

04_auditoria_llm_y_match_final

May 9, 2026

1 04. Auditoria LLM y Match Final

1.1 Goal

Audit fuzzy winners with the LLM, keep traceability of verdicts, and publish the validated entity mapping to use downstream.

1.2 Inputs

- outputs/leads_torneo_ganadores.csv
- outputs/horas_torneo_ganadores.csv
- Optional exact match outputs for reference.

1.3 Outputs

- outputs/audit_df.csv
- outputs/match_final_empresas.csv
- Optional manual override table for disputed cases.

```
[1]: # Helpers y rutas
from pathlib import Path
import os, json
import pandas as pd
import urllib.request, urllib.error

try:
    from IPython.display import display
except Exception:
    def display(x): print(x)

def find_project_root(start=None):
    start = (start or Path.cwd()).resolve()
    for p in [start, *start.parents]:
        if (p / "01_data_ingestion_enrichment").is_dir() and (p / "02_data_cleaning").is_dir():
            return p
    raise FileNotFoundError(f"No se pudo localizar la raíz. cwd: {start}")

def ensure_dir(d): Path(d).mkdir(parents=True, exist_ok=True); return Path(d)
```

```
def save_df_csv(df, path, *, index=False, encoding="utf-8-sig"):
    path = Path(path); ensure_dir(path.parent)
    df.to_csv(path, index=index, encoding=encoding)
    print(f"[OK] Guardado: {path.resolve()} shape: {df.shape}")
    return path

def read_csv_checked(path, **kwargs):
    path = Path(path)
    if not path.exists(): raise FileNotFoundError(f"No existe: {path.resolve()}")
    return pd.read_csv(path, **kwargs)

ROOT = find_project_root()
CLEAN_OUT = ROOT / "02_data_cleaning" / "outputs"
print(f"[CONFIG] ROOT : {ROOT}")
print(f"[CONFIG] CLEAN_OUT : {CLEAN_OUT}")
```

```
[CONFIG] ROOT      : E:\TESIS MAESTRIA\Desarrollo_clustering_maestria
[CONFIG] CLEAN_OUT : E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning\outputs
```

```
[2]: # Cargar ganadores del torneo
leads_torneo = read_csv_checked(CLEAN_OUT / "leads_torneo_ganadores.csv")
horas_torneo = read_csv_checked(CLEAN_OUT / "horas_torneo_ganadores.csv")

print(f"[LEADS torneo] {leads_torneo.shape} cols: {leads_torneo.columns.tolist()}")
print(f"[HORAS torneo] {horas_torneo.shape} cols: {horas_torneo.columns.tolist()}")

# También cargar los exactos para referencia
leads_exact = read_csv_checked(CLEAN_OUT / "leads_ruc_exact.csv")
horas_exact = read_csv_checked(CLEAN_OUT / "horas_ruc_exact.csv")
```

```
[LEADS torneo] (305, 6) cols: ['Company_raw', 'Company_norm', 'source_winner',
'winner_name_norm', 'score', 'RUC']
[HORAS torneo] (99, 6) cols: ['EMPRESA_raw', 'EMPRESA_norm', 'source_winner',
'winner_name_norm', 'score', 'RUC']
```

1.4 Migrate Here From Source Notebook

- Build audit_cases.
- OpenAI audit call.
- Final export logic.

1.4.1 Suggested Source Cells

- Code cells: 26, 27, 28

```
[3]: # Construir casos de auditoría
MAX_NAME_CHARS = int(os.getenv("AUDIT_MAX_NAME_CHARS","160"))
AUDIT_THRESHOLD = int(os.getenv("UMBRAL_SRI","80"))

def _clip(x, n=MAX_NAME_CHARS):
    if x is None or (isinstance(x, float) and pd.isna(x)): return ""
    s = str(x); return s if len(s)<=n else s[:n]+"..."

LLM_JUDGE_PROMPT = """Eres auditor de matching de empresas (Ecuador).
Entrada: JSON {cases:
    ↳[{id,source_raw,source_norm,candidate_norm,ruc,score,source_label}]}].
REGLA: Si ruc no tiene 13 dígitos => verdict=incorrect.
Si hay coherencia total entre source_raw y candidate_norm => correct.
Si es genérico o faltan datos => uncertain.
Salida: JSON {results:[{id,verdict,confidence,reason}]}].
confidence en [0,1]. reason: 1 frase <=80 chars. SIN saltos de línea."""

def _build_cases(df: pd.DataFrame, prefix: str) -> list:
    if df is None or df.empty: return []
    d = df.copy()
    if "score" in d.columns:
        d = d[d["score"].astype(float) >= float(AUDIT_THRESHOLD)]
    if d.empty: return []
    raw_col = "Company_raw" if "Company_raw" in d.columns else _
↳("EMPRESA_raw" if "EMPRESA_raw" in d.columns else None)
    norm_col = "Company_norm" if "Company_norm" in d.columns else _
↳("EMPRESA_norm" if "EMPRESA_norm" in d.columns else None)
    cand_col = "winner_name_norm" if "winner_name_norm" in d.columns else None
    ruc_col = "RUC" if "RUC" in d.columns else None
    cases = []
    for i, r in d.iterrows():
        src_win = str(r.get("source_winner",""))
        cases.append({
            "id": f"{prefix}-{src_win}-{i}",
            "source_label": f"{prefix}_{src_win}",
            "source_raw": _clip(r.get(raw_col,"") if raw_col else ""),
            "source_norm": _clip(r.get(norm_col,"") if norm_col else ""),
            "candidate_norm": _clip(r.get(cand_col,"") if cand_col else ""),
            "score": None if pd.isna(r.get("score")) else float(r.
↳get("score")),
            "ruc": "" if pd.isna(r.get(ruc_col,"")) else str(r.
↳get(ruc_col,"")),
        })
    return cases

audit_cases = _build_cases(leads_torneo, "LEADS") + _build_cases(horas_torneo, _
↳"HORAS")
```

```

audit_df_preview = pd.DataFrame(audit_cases)
print(f"[AUDITORIA] Casos a auditar: {len(audit_cases)}")
if not audit_df_preview.empty:
    print(audit_df_preview["source_label"].value_counts().to_string())
display(audit_df_preview.head(10))

```

[AUDITORIA] Casos a auditar: 404

source_label

LEADS_SRI_FANTASIA	154
LEADS_SCVS	78
LEADS_SRI_RAZON	73
HORAS_SCVS	39
HORAS_SRI_FANTASIA	31
HORAS_SRI_RAZON	29

	id	source_label	source_raw \
0	LEADS-SCVS-0	LEADS_SCVS	WWE
1	LEADS-SCVS-1	LEADS_SCVS	THE HOME DEPOT MÉXICO
2	LEADS-SCVS-2	LEADS_SCVS	Tesalia
3	LEADS-SCVS-3	LEADS_SCVS	Telefonica
4	LEADS-SRI_RAZON-4	LEADS_SRI_RAZON	TECNOLOGICO DE MONTERREY
5	LEADS-SRI_FANTASIA-5	LEADS_SRI_FANTASIA	TALMEX PHARMA
6	LEADS-SCVS-6	LEADS_SCVS	Talma
7	LEADS-SRI_RAZON-7	LEADS_SRI_RAZON	SYM
8	LEADS-SRI_RAZON-8	LEADS_SRI_RAZON	SUMA GLOBAL PACK
9	LEADS-SCVS-9	LEADS_SCVS	STEVE MADDEN

	source_norm	candidate_norm \
0	WWE	WORLD WIDE ELECTRIC SUPPLY WWE ECUADOR S EN LI...
1	THE HOME DEPOT MEXICO	DEPOT
2	TESALIA	THE TESALIA SPRINGS COMPANY
3	TELEFONICA	ASISTENCIA TECNICA TELEFONICA ASSISTPHONE
4	TECNOLOGICO MONTERREY	ASOCIACION CIVIL INSTITUTO TECNOLOGICO ESTUDIO...
5	TALMEX PHARMA	PHARMA
6	TALMA	TALMA ECUADOR SERVICIOS AEROPORTUARIOS
7	SYM	CONSORCIO SYM CONTABLESTRIBUTARIOS
8	SUMA GLOBAL PACK	GLOBAL
9	STEVE MADDEN	STEVE MADDEN DISTRIBUTION ECUADOR S

	score	ruc
0	100.0	1793160182001
1	100.0	991307281001
2	100.0	1790005739001
3	100.0	1792712750001
4	100.0	991463712001
5	100.0	910479948001
6	100.0	1791807162001
7	100.0	1792585376001

```
8 100.0 1792923611001
9 100.0 1793216824001
```

```
[4]: # Llamada a OpenAI para auditar
# Requiere variable de entorno OPENAI_API_KEY.
# Si no hay casos, se salta y se genera match_final directamente desde los
↳exactos.

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY", "")
if not OPENAI_API_KEY:
    try:
        import getpass
        OPENAI_API_KEY = getpass.getpass("OPENAI_API_KEY (oculto): ").strip()
    except Exception:
        OPENAI_API_KEY = ""

OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-4o-mini")
OPENAI_URL = os.getenv("OPENAI_URL", "https://api.openai.com/v1/chat/
↳completions")

def _extract_json(text: str) -> str:
    s = str(text or "").strip()
    if s.startswith("`"): s = s.split("\n",1)[1].rsplit("`",1)[0].strip()
    i,j = s.find("{"), s.rfind("}")
    return s[i:j+1] if i!=-1 and j!=-1 else s

def _call_llm(chunk: list) -> list:
    payload = {"model": OPENAI_MODEL, "temperature": 0,
               "max_tokens": int(os.getenv("OPENAI_MAX_TOKENS", "2000")),
               "response_format": {"type": "json_object"},
               "messages": [
                   {"role": "system", "content": LLM_JUDGE_PROMPT},
                   {"role": "user", "content": json.dumps({"cases": chunk},
↳ensure_ascii=False)},
               ]}
    data = json.dumps(payload).encode("utf-8")
    req = urllib.request.Request(OPENAI_URL, data=data,
                                 headers={"Content-Type": "application/json", "Authorization": f"Bearer_
↳[OPENAI_API_KEY]"}, method="POST")
    with urllib.request.urlopen(req, timeout=300) as resp:
        raw = resp.read().decode("utf-8")
        content = _extract_json(json.loads(raw)["choices"][0]["message"]["content"])
        return json.loads(content).get("results", [])

def _judge_all(cases: list) -> pd.DataFrame:
    if not cases: return pd.
↳DataFrame(columns=["id", "verdict", "confidence", "reason"])
```

```

chunk_size = int(os.getenv("AUDIT_CHUNK_SIZE", "10"))
results = []
i = 0
while i < len(cases):
    try:
        res = _call_llm(cases[i:i+chunk_size])
        results.extend(res)
        i += chunk_size
    except (json.JSONDecodeError, ValueError):
        chunk_size = max(2, chunk_size//2)
return pd.DataFrame(results)

if audit_cases and OPENAI_API_KEY:
    print(f"Auditando {len(audit_cases)} casos con {OPENAI_MODEL}...")
    judge_df = _judge_all(audit_cases)
    cases_df = pd.DataFrame(audit_cases)
    out_df = cases_df.merge(judge_df, on="id", how="left")
    print(f"\nResultados auditados: {len(out_df)}")
    if "verdict" in out_df.columns:
        print(out_df["verdict"].value_counts(dropna=False).to_string())
        _ = save_df_csv(out_df, CLEAN_OUT / "audit_df.csv")
else:
    print("[INFO] Sin API key o sin casos: se omite la auditoría LLM.")
    out_df = pd.DataFrame(audit_cases) if audit_cases else pd.DataFrame()
    if not out_df.empty:
        out_df["verdict"] = "skipped"; out_df["confidence"] = pd.NA;
    out_df["reason"] = "no api key"
    _ = save_df_csv(out_df, CLEAN_OUT / "audit_df.csv")

```

Auditando 404 casos con gpt-4o-mini...

Resultados auditados: 404

verdict

incorrect 162

correct 145

uncertain 97

[OK] Guardado: E:\TESIS

MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning\outputs\audit_df.csv

shape: (404, 10)

```

[5]: # Match final: combinar exactos + ganadores validados
# Columnas finales: source_label, name_raw, name_norm, RUC, source_winner,
    score, verdict

def _build_final_map(exact_df: pd.DataFrame, torneo_df: pd.DataFrame,
                    audit_df_: pd.DataFrame, raw_col: str, norm_col: str,
    label: str) -> pd.DataFrame:

```

```

# Exactos ya tienen RUC confiable
exact_ok = exact_df[exact_df["RUC"].notna()].copy()
exact_ok["source_label"] = label
exact_ok["name_raw"] = exact_ok[raw_col]
exact_ok["name_norm"] = exact_ok[norm_col]
exact_ok["source_winner"] = "SCVS_EXACT"
exact_ok["score"] = 100.0
exact_ok["verdict"] = "correct"
exact_ok = □
↪exact_ok[["source_label", "name_raw", "name_norm", "RUC", "source_winner", "score", "verdict"]]

# Ganadores del torneo con verdict=correct (o skipped si no hubo auditoría)
if torneo_df.empty:
    return exact_ok
rc = raw_col; nc = norm_col
torneo_merged = torneo_df.copy()
torneo_merged["source_label"] = label
torneo_merged["name_raw"] = torneo_merged[rc] if rc in torneo_merged.
↪columns else ""
torneo_merged["name_norm"] = torneo_merged[nc] if nc in torneo_merged.
↪columns else ""
if not audit_df.empty and "id" in audit_df.columns:
    # Unir veredictos. La columna id en audit_df tiene prefijo □
    ↪"{label}-{src_win}-{idx}"
    id_map = {r["id"]: r.get("verdict", "uncertain") for _, r in audit_df.
↪iterrows() if str(r.get("source_label", "")).startswith(label)}
    torneo_merged["_row_id"] = [f"{label}-{r.get('source_winner', '')}-{i}" □
↪for i, (_, r) in enumerate(torneo_df.iterrows())]
    torneo_merged["verdict"] = torneo_merged["_row_id"].map(id_map).
↪fillna("skipped")
else:
    torneo_merged["verdict"] = "skipped"

torneo_ok = torneo_merged[torneo_merged["verdict"]
↪isin(["correct", "skipped"])] .copy()
torneo_ok = □
↪torneo_ok[["source_label", "name_raw", "name_norm", "RUC", "source_winner", "score", "verdict"]]

return pd.concat([exact_ok, torneo_ok], ignore_index=True).
↪drop_duplicates("name_norm", keep="first")

audit_df_ref = pd.read_csv(CLEAN_OUT / "audit_df.csv") if (CLEAN_OUT/"audit_df.
↪csv").exists() else pd.DataFrame()

final_leads = _build_final_map(leads_exact, leads_torneo, audit_df_ref, □
↪"Company_raw", "Company_norm", "LEADS")

```

```

final_horas = _build_final_map(horas_exact, horas_torneo, audit_df_ref,
    ↪ "EMPRESA_raw", "EMPRESA_norm", "HORAS")

match_final = (
    pd.concat([final_leads, final_horas], ignore_index=True)
    .drop_duplicates(subset=["name_norm"], keep="first")
    .sort_values("name_norm")
    .reset_index(drop=True)
)

print(f"[MATCH FINAL] Total entidades mapeadas: {len(match_final)}")
print(f"    Con RUC: {int(match_final['RUC'].notna().sum())}")
print(f"    Sin RUC: {int(match_final['RUC'].isna().sum())}")

_ = save_df_csv(match_final, CLEAN_OUT / "match_final_empresas.csv")

# Verificación
for p in [CLEAN_OUT/"match_final_empresas.csv"]:
    if not p.exists(): raise FileNotFoundError(f"Output faltante: {p}")
    df_chk = pd.read_csv(p)
    print(f"[OK] {p.name} shape={df_chk.shape}")
print("\n Notebook 04_auditoria_llm_y_match_final completado correctamente.")
display(match_final.head(10))

```

[MATCH FINAL] Total entidades mapeadas: 169

Con RUC: 169

Sin RUC: 0

[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning
 \outputs\match_final_empresas.csv shape: (169, 7)

[OK] match_final_empresas.csv shape=(169, 7)

Notebook 04_auditoria_llm_y_match_final completado correctamente.

	source_label	name_raw	name_norm	RUC \
0	LEADS	7-ELEVEN MEXICO	7 ELEVEN MEXICO	1.400613e+12
1	LEADS	ABBOTT	ABBOTT	9.900007e+11
2	LEADS	Abbvie	ABBVIE	1.792489e+12
3	LEADS	AFP Genesis	AFP GENESIS	9.913076e+11
4	LEADS	Akros	AKROS	1.791149e+12
5	LEADS	Alvarez Barba S.A	ALVAREZ BARBA	1.790361e+12
6	LEADS	ANHEUSER -BUSCH INBEV	ANHEUSER BUSCH INBEV	9.931126e+11
7	HORAS	Aseguradora del Sur	ASEGURADORA SUR	1.901236e+11
8	HORAS	Asesoría y Control	ASESORIA CONTROL	1.792766e+12
9	HORAS	AVIS	AVIS	1.793120e+12

	source_winner	score	verdict
0	SRI_FANTASIA	100.0	correct
1	SCVS	100.0	correct

2	SRI_RAZON	100.0	correct
3	SCVS	100.0	correct
4	SCVS_EXACT	100.0	correct
5	SCVS_EXACT	100.0	correct
6	SRI_RAZON	100.0	correct
7	SCVS	100.0	correct
8	SCVS_EXACT	100.0	correct
9	SCVS_EXACT	100.0	correct